

Topic 10.4

10.4: Has-A and Is-A Relationships

There are two important relationships that are frequently discussed in OOD/OOP.

An *Is-A* relationship indicates that one object type is derived from another. In the homework problem regarding vehicles, airplanes, etc., *airplane* is derived from *vehicle*. Therefore, the relationship *airplane Is-A vehicle* exists.

A *Has-A* relationship indicates that one object includes another object type in its implementation. For example, in keeping track of airline flights, we might keep information such as the aircraft number, flight number, origin, destination, departure and arrival times, and a list of the passengers on board. In this case, the relationship *flight Has-A list* exists.

Recall the earlier discussion about overloading the stream insertion operator in hierarchies. In the following example, we define print functions for each level in the hierarchy. We then define a stream insertion function that takes a pointer as an argument. However, it doesn't print out the pointer "address" value, but rather uses the virtual print function to print out the specific item pointed to. We only need one version of the stream insertion operator function to handle the entire hierarchy.

```
// MODULE:      inherit9.cpp
// PROGRAMMER:   Gary J. Blair
// LANGUAGE:     C++
// DATE:        96/04/11
//
// PURPOSE:
//   Illustrate virtual functions.
//   Illustrate stream inserting of a pointer.
//

#include <iostream.h>
#include <stdlib.h>
#include <string.h>

class Thing // Thing is the base of
           // all our objects.
{
public:
    Thing() { strcpy(name,""); };
    Thing( char * thing_name )
    { strcpy(name,thing_name); };
    virtual void print() { print(cout); };
    virtual void print(ostream& str);
    friend ostream& operator <<
        ( ostream& str, Thing *t );

protected:
    char name[80];
};

void Thing::print(ostream& str)
{
    str << "name:" << name;
}
```

```

ostream& operator << ( ostream& str, Thing *t )
{
    t->print(str);
    return str;
}

class Person : public Thing
{
    public:
        Person();
        Person( char *person_name, int person_age );
        virtual void print() { print(cout); };
        virtual void print(ostream& str);

    protected:
        int age;
};

Person::Person() : Thing()
{
    age = -1;
}

Person::Person( char *person_name,
                int person_age )
    : Thing(person_name)
{
    age = person_age;
}

void Person::print(ostream& str)
{
    Thing::print(str);
    str << " age: " << age;
}

class Politician : public Person
{
    public:
        Politician( char *person_name, int person_age,
                    char *pol_party );
        virtual void print() { print(cout); };
        virtual void print(ostream& str);

    protected:
        char party[40];
};

Politician::Politician( char *person_name,
                        int person_age,
                        char *pol_party )
    : Person(person_name, person_age)
{
    strcpy(party, pol_party);
}

void Politician::print(ostream& str)
{
    Person::print(str);
    str << " party:" << party;
}

class SimpleList : public Thing
{
    public:

```

```

SimpleList() { size = 0;};
void insert( Thing& new_item ) {items[size++] =
&new_item;};
virtual void print() { print(cout); };
virtual void print( ostream& str );

protected:
int size;
Thing* items[50];
};

void SimpleList::print( ostream& str )
{
    for (int i=0; i<size; ++i)
        str << items[i] << endl;
}

void main(void)
{
    Thing      t("something"),x("another");
    Person     me("Gary J. Blair",32);
    Person     wife("Tammy Blair",34);
    Politician pres("George Bush",68,"Republican");
    Politician vp("Dan Quayle", 48, "Republican");
    SimpleList main_list, sub_list;

    main_list.insert(t);
    main_list.insert(me);
    main_list.insert(pres);
    cout << "Printing main list.\n"
        << (Thing *) &main_list
        << "\n";
    sub_list.insert(x);
    sub_list.insert(wife);
    sub_list.insert(vp);
    cout << "Print sub-list.\n"
        << (Thing *) &sub_list
        << "\n";

    main_list.insert(sub_list);
    cout <<
        "Printing main list containing sub-list.\n"
        << (Thing *) &main_list
        << "\n";
}

```

Output of inherit9.cpp:

```

Printing main list.
name:something
name:Gary J. Blair age: 32
name:George Bush age: 68 party:Republican

Print sub-list.
name:another
name:Tammy Blair age: 34
name:Dan Quayle age: 48 party:Republican

Printing main list containing sub-list.
name:something
name:Gary J. Blair age: 32
name:George Bush age: 68 party:Republican
name:another

```

name:Tammy Blair age: 34
name:Dan Quayle age: 48 party:Republican

[Back to Module 10 \(https://onlinelearning.berkeley.edu/courses/2291660/pages/module-10\)](https://onlinelearning.berkeley.edu/courses/2291660/pages/module-10)